CODER — User Manual

CODER is a programmable module: you type a short program into the module, and the program becomes the patch. Step sequencers, effects, modulation sources, and small realtime DSP structures are all written the same way — a few plain lines. No programming experience is assumed: if you can patch a modular, you can read every example in this manual.

This is the manual for the rescue build (2026). CODER originally shipped in 2019 with a JavaScript engine inside; that engine is gone. In its place is a small language designed for modular work. Programs are short — most of this manual's examples are one line — and the module is built to be forgiving: a typo never stops the sound. If your program has a mistake, the last good program keeps playing and the error is written on the panel, in plain words, with its line number.

Quick start

A fresh CODER already contains a starter program. It plays immediately — CODER keeps its own time when no clock is patched:

```
# SETUP
let seed = 1
let tempo = 120

# PLAY
let bass = semi([0 0 12 0 3 0 10 7])
out1 = bass
out2 = gate(spread(3, 8))
```

- **OUT 1** carries a little bassline (stepped pitch CV).
- **OUT 2** fires gates in a 3-against-8 pattern.
- Patch OUT 1 to a VCO's V/OCT input and OUT 2 to an envelope's gate input

and you have a voice. Patch a clock into IN 1 and CODER follows it instead of its internal tempo.

Your first one-line program, in the PLAY box, is just:

```
out1 = in1
```

—a patch cord. Everything else in this manual is a variation on that line.

The two boxes

SETUP (top) runs once, when your program loads. Definitions live there: patterns you want to name, the tempo, the random seed, the clock choice.

PLAY (bottom) is the live program: the lines that wire outputs.

Both boxes use the same language. Editing either box recompiles the whole program; the new program takes over only if it is completely correct.

Lines

One statement per line. A line is one of:

- `let name = ...` — define something and give it a name.
- `out1 = ...` (also `out2`, `out3`, `out4`; the old spellings `output1` … `output4` still work) — wire an output jack.

starts a comment, anywhere on a line.

Values

Programs work with three kinds of values:

- **Numbers** — plain values: `0.5`, `5`, `-2.3`. In this module, numbers are volts unless a construct says otherwise.
- **Signals** — streams that flow per sample: jack inputs, oscillators, filter outputs. Signal math is continuous: `in1 * 0.5` is a VCA.
- **Patterns** — looping lists of steps. A pattern used in a line reads as its current step's value, held until the next step — stepped CV.

The clock

Patterns advance on the clock: the gate on IN 1 by default. If nothing is patched into the clock input, CODER uses its internal clock (120 BPM, or whatever `let tempo = ...` says in SETUP). To move the clock: `let clock = in2` in SETUP. Patterns of different lengths share the clock, so polymeter is automatic. `phase` reads 0→1 inside the current step; `beat` counts steps.

Errors can never hurt you

- A bad line changes nothing audible. The previous good program keeps running; the Error LED lights and the message appears under the PLAY box: `Line 3: I don't know 'lp2'. Did you mean 'lp()'?`
- There are no loops and no way to write a program that hangs: a program's size is capped, and every value is sanitized as it flows (see Limits).

Language reference

Every construct below has an example you can paste into PLAY as-is. Unless noted, examples assume nothing is in SETUP.

Inputs

Name .

```
out1 = phase * 5            # a ramp that restarts every clock step
```

Wiring and naming

```
let bass
```

```
out1 = bass                 # wire a jack
out2 = in1 * 0.5            # or wire an expression directly
out3 = 2.5                  # a bare number is a constant voltage
```

The patch-cord arrow: `->`

Nested brackets read inside-out, which is nobody's favorite. The arrow `->` lets a line read left to right, the way a patch cord runs: whatever is on the left flows into the first input of the word on the right.

```
out1 = in1 -> lp(900) -> drive(0.3)
```

is exactly `drive(lp(in1, 900), 0.3)`, written the way you patch it. Arrows chain, and they bind looser than arithmetic: `in1 + in2 -> lp(900)` filters the sum.

Patterns

A pattern is a bracketed list of steps. Values are volts. A `-` standing alone is a rest; a minus stuck to a number is a negative number (`[0 -2]` steps 0 then −2; `[0 - 2]` steps 0, rest, 2). A `_` is a tie: the step holds — a pitch stays, and a gate stays high through the tied step instead of re-pulsing, so `[1 _ 0 0]` is one long gate, not two. Patterns can be up to 64 steps.

```
out1 = semi([0 0 12 0 3 0 10 7])   # write melodies in semitones with semi()
out1 = [0 0 2.5 0]                 # or in raw volts
```

Pattern verbs (each returns a new pattern; the original keeps playing its own course, so a pattern and its transforms advance in lockstep):

```
out1 = gate(spread(3, 8))          # spread(hits, steps): 3 hits over 8 steps
out1 = gate(flip([1 0 0 1]))       # flip(p): play it backwards
out1 = gate(shift([1 0 0 0], 1))   # shift(p, n): rotate by n steps
out1 = slow([0 7], 2)              # slow(p, n): each step lasts n clocks
out1 = fast([0 3 5 7], 2)          # fast(p, n): skip through, n steps per clock
out1 = gate(thin([1 1 1 1], 0.5))  # thin(p, amt): drop steps at random (seeded — same every load)
out1 = pick([0 0 12 0], [0 5 7 0]) # pick(p, p, …): a fresh choice of pattern each cycle
out2 = gate(spread(3, 8))          # gate(p): 5 V pulses, high for the first half of each sounding step
```

Pitch

```
out1 = qu
```

```
out1 = quant(in1, minor)           # or to a scale: major, minor, penta, dorian
out1 = semi(7)                     # semitones to volts: semi(n) = n / 12
```

(quantize ties go to the lower note.)

Effects and signal shaping

Effects are words. Each keeps its own state inside the module; chain them with ordinary lines.

```
out1 = lp(in1, 900)                    # lp(x, hz): one-pole lowpass
out1 = hp(in1, 300)                    # hp(x, hz): one-pole highpass
out1 = reso(in1, 800, 6)               # reso(x, hz, q): resonant filter
```

```
out1 = echo(in1, 0.375, 0.35, 0.3)         # echo(x, sec, feedback, mix): feedback delay
out1 = drive(in1, 0.5)                     # drive(x, amt): soft saturation, amt 0..1
out1 = fold(in1, 0.4)                      # fold(x, amt): wavefolder
out1 = crush(in1, 0.6)                     # crush(x, amt): bit + sample-rate reduction
out1 = clip(in1, -2, 2)                    # clip(x, lo, hi): hard limits
```

Filter frequencies (and most other arguments) can themselves be signals:

```
out1 = lp(in1, 400 + lfo(0.3, sine) * 60)  # a filter that breathes
```

Realtime DSP from primitives

`tap` reads a signal delayed; `z` is a signal's value one sample ago. A signal may refer to itself only through `z(...)` or `tap(...)` — that single rule is what makes feedback safe to write, and it is enough to build real DSP structures from raw algebra:

```
let y = in1 * 0.2 + z(y) * 0.8             # a one-pole filter, by hand
out1 = y
```

```
# a lowpass written out as a biquad (1000 Hz, Q 0.71, at 48 kHz)
let y = 0.00392*in1 + 0.00783*z(in1) + 0.00392*z(z(in1)) + 1.8154*z(y) - 0.8310*z(z(y))
out1 = y
```

```
let comb = in1 * 0.5 + tap(comb, 0.011) * 0.6   # a feedback comb, by hand
out1 = comb
```

Delays run up to 2 seconds. Feedback amounts are clamped inside the runtime, so a structure like this can never run away.

Modulation and utilities

```
out1 = l
```

```
out1 = noise()                      # white noise, ±5 V (seeded: same every load)
out1 = slew(in1, 0.2)               # slew(x, sec): glide — 0.2 s to swing 5 V
out1 = sah(in2, trig1)              # sah(x, trigger): sample & hold
out1 = env(gate(spread(2, 8)), 0.01, 0.4)   # env(trigger, attackSec, decaySec): 0–5 V
out1 = follow(in1, 0.05)            # follow(x, sec): envelope follower
out1 = rand()                       # a new random voltage (0–5) every clock step
out1 = walk(0.4)                    # a random-walk voltage, moves on every clock step
out1 = loop(8, 0.85)                # loop(steps, lock): a random sequence that repeats;
                                    # each cycle, steps re-roll with chance (1 − lock).
                                    # lock 1 = frozen forever, 0 = new every cycle
out1 = cycle(in1, in2, in3)         # a sequential switch: steps through its inputs
                                    # on every clock step (all inputs keep flowing)
```

Two idioms worth knowing:

```
out1 = in1 * clamp(1 - follow(in2, 0.3) / 5, 0, 1)   # ducking: in2's loudness turns in1 down
out2 = gate(slow([1], 4))                            # a clock divider: one gate every 4 steps
```

Triggers fire as a signal rises past 1 V.

Math

`+` `-` `*` `/` `%` and parentheses, plus:

```
out1 = abs(in1)                     # rectification
```

```
out1 = min(in1, in2)              # the lower of two signals
out1 = max(in1, 0)                # half-wave rectification
out1 = clamp(in1, -1, 1)          # keep a signal inside bounds
out1 = range(in1, 0, 10)          # rescale the standard −5..5 range to 0..10
out1 = map(in1, -5, 5, 0, 10)     # rescale any range to any other, explicitly
out1 = mix(in1, in2, 0.25)        # crossfade, t 0..1
out1 = sin(phase)                 # sin() works in cycles: sin(0.25) = 1
```

Division by zero gives 0. Nothing in CODER can produce an infinite or not-a-number value — see Limits.

SETUP directives

In SETUP only:

```
let tempo = 132        # internal clock speed (20–400), used when no clock is patched
let clock = in2        # take the pattern clock from another jack
let seed = 7           # the random seed: same seed + same program = same music
```

Tutorial: first sound to a full patch

1. One line — a gated tremolo. Patch audio into IN 1, a gate into IN 2:

```
out1 = in1 * gate2
```

2. Two lines — give it movement.

```
out1 = lp(in1, 700 + lfo(0.25, sine) * 80) * gate2
```

3. Three lines — CODER becomes the sequencer. Unpatch the gate; patch OUT 2 to your envelope instead:

```
let bass = semi([0 0 12 0 3 0 10 7])
out1 = bass
out2 = gate(spread(3, 8))
```

Patch OUT 1 to a VCO's V/OCT and OUT 1's voice audio back into IN 1, and step 2's filter line now processes your sequence.

4. A full patch (10 lines). A small self-playing system: bassline, hats made of shaped noise, a wandering modulation, and an echoed copy of the bass voice (patch the voice's audio into IN 1):

```
# SETUP
let seed = 3
let tempo = 132

# PLAY
let bass = semi([0 0 12 0 3 - 10 7])
let hats = spread(5, 8)
out1 = quant(bass, minor)
out2 = gate(spread(3, 8))
out3 = env(gate(hats), 0.002, 0.05) * noise() * 0.2
out4 = echo(lp(in1, 900), 0.341, 0.35, 0.3)
```

Every line of that is a patching idea you already know, written down.

Limits and guarantees (the containment law)

- **Size:** a program may use at most 512 parts (each word like `lp(...)`
is a part). Patterns: at most 64 steps. Delays: at most 2 seconds each. Measured on a typical machine, ordinary programs — sequencers, effect chains, hand-built filters — use about 1% of one processor core. The expensive parts are `reso(...)` and `sin(...)`: chaining a hundred of those in a row will eat most of a core, so keep heavy parts in the dozens. The 512-part cap is a safety bound, not a performance promise.
- **Safety:** every flowing value is sanitized — not-a-number and infinity
become 0; outputs are clamped to ±10 V; feedback states that decay toward nothing are flushed to exactly zero (no denormal slowdowns). A badly behaved program degrades to silence on the offending part — never a runaway, never a crash, never a frozen host.
- **Determinism:** randomness is seeded. Same program + same seed + same
inputs ⇒ the same output, every load. Patterns restart at step 1 when a preset loads.
- **Presets:** your program text (both boxes) is saved with the preset and
restored with it.
- **Sample rate:** `sr` and all delay/filter math use the host's rate at
load time.
- A signal only flows while it is wired to something (directly or through a
`let` that is). Unused definitions cost nothing.

Coming from the old JavaScript CODER?

The old box ran JavaScript on gate edges. Old scripts do not run in this build — deliberately: the JavaScript engine was the fragile part, and it is gone. The translation is small, because the new language keeps the old names where it counts:

```
Old (JavaScript)   ·   Now
                                                                `output1
```

The old module also ran your script only when a gate changed. The new one runs continuously — which is why effects and audio-rate DSP are possible at all.

Changelog — rescue build (2026)

- New language and runtime replace the JavaScript engine (see above).
- Errors are shown on the panel in plain words, with line numbers; a bad
program never interrupts the current sound.
- Programs pre-load with a commented starter example; editor boxes use a
monospace font; boxes are captioned SETUP / PLAY; jacks IN 1–4 / OUT 1–4.
- Program text is saved in presets explicitly, and restored on load.

- Full manual published with the module.

Acknowledgments

The design of this language was informed by the live-coding tradition — TidalCycles' patterns-as-values, Sonic Pi's plain-language teaching, and ChucK's respect for musical time are acknowledged with thanks. The language, its vocabulary, and this implementation are original to BenDavis.IO; no syntax or code was taken from any of them.